

캡스톤 프로젝트 발표

팀 체리피커

김한별 박재행 이명빈 조준하

목차

- 배경 및 문제 정의
- 데이터 정제
- 시스템 아키텍처 및 구현
- 의의 / 한계
- 질의응답

문제

문제

- 1인당 평균 신용카드 보유 4.6장
- 혜택 정보를 외우거나 한번에 찾기 쉽지 않음
- 결제 시 최적 카드 선택에 어려움 존재

해결방안

- 데이터 구조화 + 위치 기반으로 문제 해결
- 소비 상황 별 최적화된 카드 사용

이종 데이터 수집 및 통합

소상공인시장진흥공단 DB: 전국 250만 매장, 위도·경도 포함
카드고릴라 크롤링: 약 3000개 카드 혜택 텍스트
비정형 → 정형화 필요

```
for i in TARGET_RANGE:
    if i % 50 == 0:
        driver.quit()
        driver = create_driver()

    url = f'https://www.card-gorilla.com/card/detail/{i}'
    try:
        driver.get(url)
        # 카드 이름
        try:
            WebDriverWait(driver, 30, until=
                EC.presence_of_element_located((By.CSS_SELECTOR, "strong.card")))
        except:
            print(f'{{{i}}} ✖ 페이지 로딩 실패')
            continue

        card_name = driver.find_element(By.CSS_SELECTOR, "strong.card").text.strip()
        # 카드 혜택
        try:
            buttons = driver.find_elements(By.CSS_SELECTOR, "div.bnf:first dt, article.ond.com.benefit dt, section.coupon dt")
            for btn in buttons:
                try:
                    driver.execute_script('arguments[0].click();', btn)
                    time.sleep(0.1) # 리프팅에 간격
                except: pass

            time.sleep(1.0) # 스크롤에 필요하기 때문에 대기
            soup = BeautifulSoup(driver.page_source, 'html.parser') # 데이터 추출

            # 혜택 영역
            benefit_area = soup.select_one("div.bnf:first, article.ond.com.benefit")

            if benefit_area:
                parse_universal_content(benefit_area)
                raw_data = benefit_area.get_text(separator="\\n", strip=True)
            else:
                raw_data = "해당 카드 없음"
```

(카드고릴라 Crawler의 일부)

	A	B	C	D	E	
1	상가업소번호상호명	경도	위도	카드혜택_분류		
2	MA010120	백반/한정	128.59207	35.246696	외식	
3	MA010120	늘숨창의마	128.08349	34.941834	교육/학원	
4	MA010120	플펜션	128.91891	35.28639	문화/여가/숙박	
5	MA010120	용정관	128.56421	35.213857	외식	
6	MA010120	이현	128.07925	35.196188	외식	
7	MA010120	아모르펜션	127.78165	35.801612	문화/여가/숙박	
8	MA010120	함안면종합	128.42299	35.242665	뷰티/생활서비스	
9	MA010120	공단녹즙	129.04922	35.364265	쇼핑	
10	MA010120	스테이수출	128.3211	34.885484	문화/여가/숙박	
11	MA010120	스캐폴딩양	128.05928	35.171569	교육/학원	
12	MA010120	에코맨	128.58283	35.238778	쇼핑	
13	MA010120	웰컴돈.막	128.64377	34.894599	외식	
14	MA010120	도원호프	128.59274	35.167628	외식	

(매장 DB의 가공 전 원본)

LLM 기반 구조화 파이프라인

LangChain + Gemini 2.5 Flash
PydanticOutputParser로 JSON 강제
API Key 로테이션 + 증분 저장으로 안정성 확보



LangChain 기반 매칭 칼럼 생성

‘스타벅스 강남점’ ↔ ‘스타벅스’, ‘전국 편의점 10% 할인’ ↔ ‘GS25’ 같은 매칭 불가 문제
LLM이 혜택 텍스트를 분석

특정 프랜차이즈 → Brand_Key | 범용 혜택 → Category_Key

```
# 데이터 모델 Pydantic 구조
class CardBenefitDetail(BaseModel):
    merchant_name: str = Field(description="상호명 (예: 스타벅스, 이마트). 특정 상호가 없으면 업종명(예: 대중교통).")
    category: str = Field(description="대분류 (예: 푸드, 쇼핑, 교통, 주유, 통신, 공과금, 여행 등)")
    benefit_summary: str = Field(description="혜택 요약 (예: 10% 할인, 리터당 60원 적립)")
    conditions: str = Field(description="실적 조건. 없으면 '없음'")
    limit_info: str = Field(description="한도/횟수. 없으면 '제한없음'")
    is_location_based: bool = Field(description="지도에 찍을 수 있는 오프라인 장소인지 여부 (True/False)")

class CardBenefitList(BaseModel):
    benefits: List[CardBenefitDetail]
```

(칼럼 검증을 위한 Pydantic 구조)

```
# 혜택 분석
analyzer_template = """
당신은 혜택 분석가입니다.
[Cleaned Text]를 읽고, 어떤 혜택들이 있는지 목록으로 정리하세요.

[할 일]
1. 텍스트에 숨겨진 모든 혜택을 하나씩 분리하여 나열하세요.
2. 각 혜택의 **대상 가맹점(Merchant)**과 **혜택 내용(Summary)**과 **조건(Condition)**을 명확히 파악하세요.
3. 아직 JSON으로 만들지 말고, **사람이 읽기 쉬운 텍스트 리스트** 형태로 출력하세요.

[Cleaned Text]:
{cleaned_text}

[Benefit List]:
"""
```

(혜택 분석 프롬프트)

이원화된 매칭 전략

- Brand_Key: 지점 제거·표준화
- Category_Key: 19개 카테고리 정의
- Few-shot 기반 분류

전체 시스템 아키텍처

Android(Kotlin)
Node.js + Express
SQLite + WAL 모드



하이브리드 공간 검색 알고리즘

1차 Bounding Box로 후보 필터링
2차 Haversine 거리 계산
정확도 + 속도 확보

```
function fetchNearbyStores(db, { latitude, longitude, radius = 500, categories = [] }) {
  requireDb(db);

  const lat = Number(latitude);
  const lon = Number(longitude);
  const rad = Number(radius);

  if (isNaN(lat) || isNaN(lon) || isNaN(rad)) {
    return [];
  }

  // Bounding box calculation for initial SQL filtering
  // 1 degree latitude is approximately 111,320 meters
  const latDelta = rad / 111320;
  // 1 degree longitude depends on latitude: 111,320 * cos(lat)
  // We use the larger delta (at the pole-ward side of the box) or just the center for approximation
  // Using center latitude for longitude delta approximation is usually sufficient for small radii
  const lonDelta = rad / (111320 * Math.cos(toRad(lat)));

  const minLat = lat - latDelta;
  const maxLat = lat + latDelta;
  const minLon = lon - Math.abs(lonDelta);
  const maxLon = lon + Math.abs(lonDelta);

  let query = `
    SELECT id, name, branch, address, longitude, latitude, source_category, normalized_category
    FROM merchants
    WHERE latitude BETWEEN ? AND ?
    AND longitude BETWEEN ? AND ?
  `;
}
```

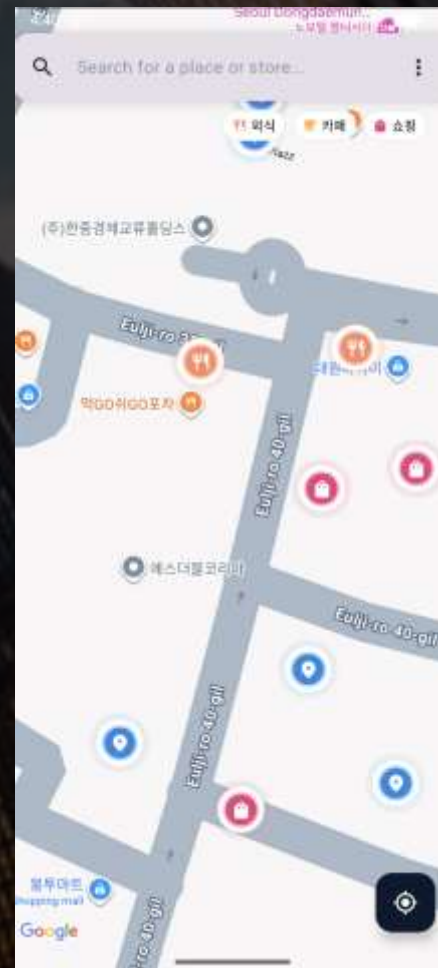
(Bounding Box를 계산하는 함수의 일부)

```
function haversineDistance(lat1, lon1, lat2, lon2) {
  const R = 6371000; // Earth radius in meters
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}
```

(Haversine 거리 계산 알고리즘)

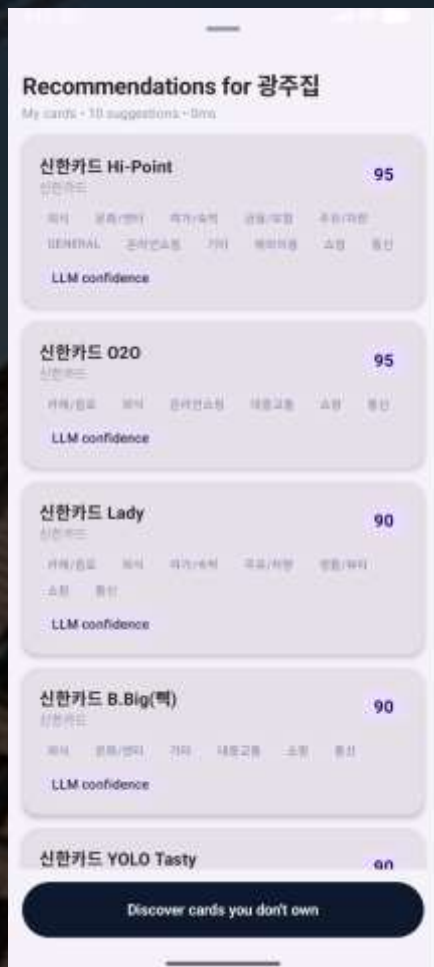
앱 구현 결과 - 지도 시각화

마커 클러스터링
카테고리 아이콘 렌더링



앱 구현 결과 - 추천 및 스트리밍

SSE 기반 실시간 추천
태그로 판단 근거 제공

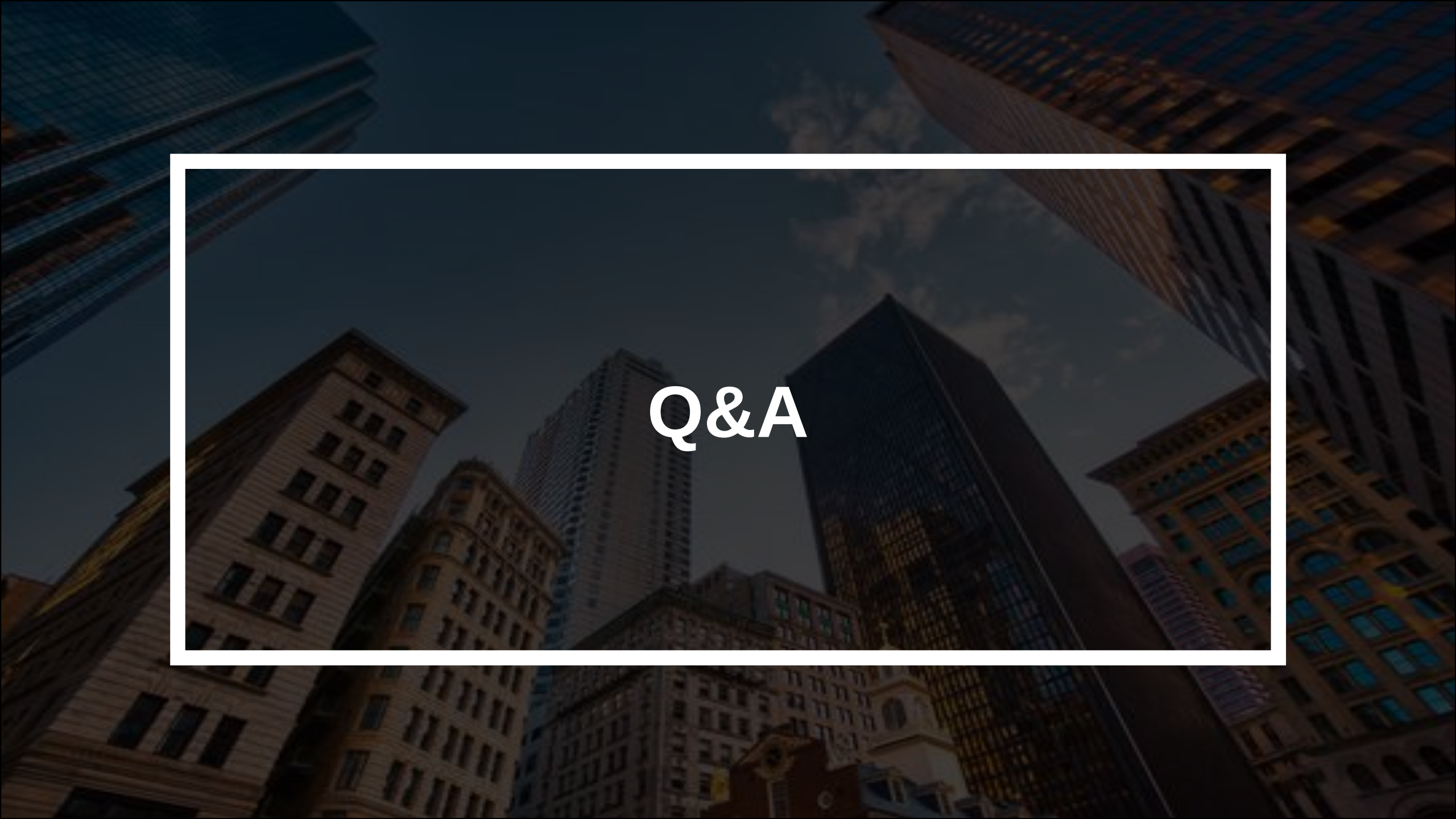


연구 의의 및 성과

- 비정형 → 정형 데이터 자산화
- 오프라인 위치 결합 서비스 구현
- 금융 소비자 편의성 강화

한계점 및 향후 과제

- 혜택 변경 실시간 반영 자동화
- 마이데이터 연동

A low-angle, upward-looking photograph of several tall skyscrapers in a city, likely New York City. The buildings are made of various materials, including glass and stone, and are set against a clear blue sky with some light clouds. The perspective creates a sense of height and scale.

Q&A